test_first/01/spec/support/system.rb
```
RSpec.configure do |config|
  config.before(:each, type: :system) do
    driven_by :rack_test
  end
end
```

This tells RSpec how to run end-to-end system tests. Specifically, you'll use a tool called rack_test, which is provided by Capybara to simulate a browser DOM tree without JavaScript. Many more details on driven_by may be found in Chapter 8, *Integration Testing with Capybara and Cucumber*, on page 151.

The first test covers the case where a user adds a project to the system. This task will be very close to Rails boilerplate, so your end-to-end test actually won't need much augmentation from unit tests. Later in this tutorial you'll add features that need more business logic.

Let's plan out what this test needs in terms of given/when/then:

- Given: You're starting with empty data, so no setup
- When: Filling out a form with project data and submitting
- Then: Verifying that the new project shows up on your list of projects with the entered tasks attached

This is what the test looks like, using the system tests added in Rails 5.1 and RSpec 3.7:

test_first/01/spec/system/add_project_spec.rb
```
Line 1 require "rails_helper"
-
- RSpec.describe "adding a project", type: :system do
-   it "allows a user to create a project with tasks" do
5     visit new_project_path
-     fill_in "Name", with: "Project Runway"
-     fill_in "Tasks", with: "Choose Fabric:3\nMake it Work:5"
-     click_on("Create Project")
-     visit projects_path
10    expect(page).to have_content("Project Runway")
-     expect(page).to have_content(8)
-   end
- end
```

This is called an "outside" test because it works from outside the Rails stack to define functionality. You're simulating user activity and browser requests and evaluating browser responses as HTML. This test is not dependent on your code's structure.